

ia-26/esgoto.py

```
1 import numpy as np
2
3 from GA import genetic_algorithm
4
5
6 flows = np.array([
7     2.5, 5.0, 7.5, 10.0, 12.5,
8     15.0, 17.5, 20.0, 22.5, 25.0
9 ]) / 1000 # m³/s
10
11 lengths = np.array([
12     20, 54, 98, 120, 34,
13     12, 88, 122, 33, 40
14 ])
15
16 diameters = np.array([
17     0.150,
18     0.200,
19     0.250,
20     0.300,
21     0.400
22 ])
23
24 costs = np.array([
25     65,
26     98,
27     150,
28     210,
29     340
30 ])
31
32 # Constantes
33 n = 0.013
34 S = 0.005
35
36 def manning_flow(D):
37     A = (
38         np.pi * D**2
39     ) / 4
40
41     Rh = D / 4
42
43     Qmax = (
44         (1 / n)
45         * A
46         * (Rh ** (2/3))
47         * np.sqrt(S)
48     )
49
50
```

```
51     return Qmax
52
53
54 def objective_function(solution):
55
56     # Converte para inteiro
57     solution = np.round(solution).astype(int)
58
59     # Garante limites
60     solution = np.clip(
61         solution,
62         0,
63         len(diameters) - 1
64     )
65
66     # Seleciona tubos
67     selected_diameters = (
68         diameters[solution]
69     )
70
71     selected_costs = (
72         costs[solution]
73     )
74
75     # Custo total
76     total_cost = np.sum(
77         selected_costs * lengths
78     )
79
80     # Penalização
81     penalty = 0
82
83     for i in range(len(solution)):
84
85         D = selected_diameters[i]
86         Q = flows[i]
87         Qmax = manning_flow(D)
88
89
90         # Restrição:
91         # Q <= 75% da capacidade
92         if Q > 0.75 * Qmax:
93             penalty += 1e9
94
95     return total_cost + penalty
96
97
98
99 if __name__ == "__main__":
100
101     best_position, best_value, history = (
102         genetic_algorithm(
103             objective_function=objective_function,
```

```

104         dim=10,
105         population_size=50,
106         generations=100,
107         bounds=(0, 4),
108         crossover_rate=0.9,
109         mutation_rate=0.1
110     )
111 )
112
113 # Converte solução final
114 best_position = np.round(
115     best_position
116 ).astype(int)
117
118 best_position = np.clip(
119     best_position,
120     0,
121     4
122 )
123
124
125 # Resultado
126 print("\n===== MELHOR SOLUÇÃO =====\n")
127
128 total_cost = 0
129
130 for i in range(10):
131
132     d = diameters[
133         best_position[i]
134     ]
135
136     c = costs[
137         best_position[i]
138     ]
139
140     trecho_cost = (
141         c * lengths[i]
142     )
143
144     total_cost += trecho_cost
145
146     print(
147         f"Trecho {i+1:2d}"
148         f" | D = {int(d*1000):3d} mm"
149         f" | Custo = R$ {trecho_cost:.2f}"
150     )
151
152 print("\n=====")
153
154 print(
155     f"\nCusto total: "
156     f"R$ {total_cost:.2f}"

```

```

157 )
158
159 print(
160     f"\nFitness final: "
161     f"{best_value}"
162 )

```